

五场景对比表格与差异分析报告

正式计算采用 `cpu\_core/...` 行的有效计数。当前测试平台为 Intel 混合架构 CPU, `cpu\_atom/...` 在本次锁核结果中多为 `` 或 ``，因此未纳入计算。

1. 五场景对比表格

1.1 指标说明

指标	计算方式	说明
IPC	<code>`instructions / cycles`</code>	每周期退休指令数，反映流水线整体利用率
L1 DCache Miss Rate（题目口径）	<code>`L1-dcache-load-misses / cache-references`</code>	题目要求的公式，保留用于提交对齐
LLC Miss Rate（题目口径）	<code>`LLC-load-misses / cache-references`</code>	题目要求的公式，保留用于提交对齐
分支预测失败率	<code>`branch-misses / branch-instructions`</code>	分支预测失败占全部分支指令比例
TLB Miss Rate（题目口径）	<code>`dTLB-load-misses / cache-references`</code>	题目要求的公式，保留用于提交对齐
Generic Cache Miss Rate	<code>`cache-misses / cache-references`</code>	perf 通用 cache 事件的 miss 比例
L1D Load Miss Rate	<code>`L1-dcache-load-misses / L1-dcache-loads`</code>	更严格的 L1D load miss rate
LLC Load Miss Rate	<code>`LLC-load-misses / LLC-loads`</code>	更严格的 LLC load miss rate
dTLB Load Miss Rate	<code>`dTLB-load-misses / dTLB-loads`</code>	更严格的 dTLB load miss rate
Frontend Bound	<code>`perf stat -M tma_frontend_bound`</code>	前端取指、解码、指令供给瓶颈比例
Backend Bound	<code>`perf stat -M tma_backend_bound`</code>	后端执行、访存、资源等待瓶颈比例

说明：题目口径中的 `cache-references` 是 perf 的通用 cache 事件，在本机并不等价于 L1D/LLC/dTLB 的访问总次数。

1.2 cache-references 事件口径排查

在第一次整理表格时，出现 L1-dcache-load-misses 远大于 cache-references。查阅资料发现对于 cache-references 的解释有两种，一种是总的缓存访问次数，一种是这个和 CPU 的具体型号有关，推测 cache-references 在本机的架构中并不是缓存总访问次数。

首先使用 `perf list --details cache-references` 和 `/sys` 中的 PMU 事件定义检查本机映射：

`perf list --details cache-references`

```
cat /sys/bus/event_source/devices/cpu_core/events/cache-references
cat /sys/bus/event_source/devices/cpu_core/events/cache-misses
```

本机输出显示：

```
cpu_core/cache-references/: event=0x2e,umask=0x4f
cpu_core/cache-misses/: event=0x2e,umask=0x41
```

这说明在当前 Raptor Lake-HX 平台上，`cache-references` 会被内核映射成 `cpu\_core` PMU 的具体硬件事件，而不是一个抽象的“所有缓存层级访问总数”。同时，`perf list` 中存在独立的 L1D 成对事件：

```
cpu_core/L1-dcache-loads/
cpu_core/L1-dcache-load-misses/
```

随后使用短时间测试交叉验证：

```
taskset -c 2 perf stat \
-e cpu_core/cache-references/,cpu_core/cache-misses/,cpu_core/L1-dcache-loads/,cpu_core/L1-
dcache-load-misses/ \
-- stress-ng --cpu 1 --cpu-method int64 -t 3s
```

该测试中观察到：

```
cache-references      = 363,397
cache-misses          = 182,952
L1-dcache-loads       = 19,597,293
L1-dcache-load-misses = 197,579
```

如果 `cache-references` 是所有缓存访问总数，它不应明显小于 `L1-dcache-loads`。但实测中 `cache-references` 比 `L1-dcache-loads` 小了几十倍，说明它不能作为 L1D load 总次数，也不能作为所有缓存访问总次数。

因此，保留题目要求的 `L1-dcache-load-misses / cache-references`、`LLC-load-misses / cache-references`、`dTLB-load-misses / cache-references` 指标；同时补充 `L1-dcache-loads`、`LLC-loads`、`dTLB-loads` 作为对应分母，计算更严格的成对 miss rate 来进行分析。

1.3 题目要求衍生指标

负载场景	IPC	L1 DCache Miss Rate (题目口径)	LLC Miss Rate (题目口径)	分支预测失败率	TLB Miss Rate (题目口径)
纯计算（整数 int64）	3.054	103.466%	0.448%	0.103%	0.146%
纯计算（浮点/矩阵 matrixprod）	2.319	959021.307%	2.862%	0.145%	0.073%

访存密集型 (read64)	4.234	158.449%	0.370%	0.000%	10.696%
随机访存型 (rand-set)	4.881	1.075%	0.161%	0.001%	0.006%
分支密集型 (queens)	1.950	113.553%	0.860%	11.625%	0.107%

1.4 补充成对 miss rate 与 Topdown 指标

负载场景	Generic Cache Miss Rate	L1D Load Miss Rate	LLC Load Miss Rate	dTLB Load Miss Rate	Frontend Bound	Backend Bound
纯计算 (整数 int64)	13.061%	1.473%	1.118%	0.00224%	0.8%	46.3%
纯计算 (浮点/矩 阵 matrixprod)	19.652%	49.372%	10.210%	0.00000%	26.3%	10.2%
访存密集 型 (read64)	10.418%	0.039%	9.752%	0.00261%	7.9%	23.2%
随机访存 型 (rand- set)	47.014%	0.015%	35.602%	0.00009%	13.8%	29.4%
分支密集 型 (queens)	14.161%	0.006%	2.195%	0.00001%	10.3%	7.7%

1.5 主导瓶颈概览

负载	执行内容	主导微架构压力	指标体现
int64	64 位整数运算	整数 ALU、后端执行 吞吐	IPC 较高, Frontend Bound 最低, Backend Bound 最高
matrixprod	矩阵乘法	浮点/SIMD、L1D 数 据供给、前端指令供 给	IPC 低于 int64, L1D Load Miss Rate 最 高, Frontend Bound 最高
read64	1G 大块内存连续读	顺序访存、硬件预	IPC 较高, 分支失败

	取	取、LLC/内存层级、TLB	率近 0, Backend Bound 中等
rand-set	512M 内存区域随机设置/访问	随机访存、cache 局部性差、预取失效	Generic Cache Miss Rate 和 LLC Load Miss Rate 最高, Backend Bound 较高
queens	N 皇后搜索、递归回溯	分支预测、bad speculation、流水线清空	IPC 最低, 分支预测失败率最高

2. 差异分析报告

1. 纯计算整数负载 int64

`int64` 主要执行 64 位整数运算，这些操作由 ALU 进行操作，如果流水线效率高，则每个 cycle 执行多条整数 μops 。它的 IPC 为 3.054，说明每个周期能够退休较多指令，说明整数运算循环具有较好的流水线利用率。

从前端看，int64 的核心循环通常由 add/sub/xor/shift/imul 等整数指令组成，代码体积小、指令循环短，因此 CPU 不需要频繁从内存取指令，L1D Load Miss Rate、LLC Load Miss Rate、dTLB Load Miss Rate 很低，负载大部分数据停留在寄存器或 L1/L2 cache 中；分支规律性强，分支预测压力较低，它的分支预测失败率只有 0.103%，说明控制流几乎不会频繁打断流水线。

前端大多数时候可以稳定给后端供应 μops ，从后端看，该负载主要压力集中在整数 ALU、整数乘法器、移位单元和执行端口调度。Topdown 中 Backend Bound 为 46.3%，明显高于 Frontend Bound 的 0.8%，说明瓶颈主要不是取指/解码，而是后端执行资源、指令依赖链或整数执行端口竞争。

2. 纯计算浮点/矩阵负载 matrixprod

matrixprod` 执行矩阵乘法，包含矩阵元素加载、乘法、加法或乘加运算。matrixprod 的 IPC 为 2.319，低于 int64，说明浮点/矩阵类指令对执行资源和数据供给的要求更高。由于分支很少，所以分支预测失败率很低。

成对指标显示，`matrixprod` 的 L1D Load Miss Rate 达到 49.372%，是五类负载中最高的；LLC Load Miss Rate 为 10.210%，也高于整数和分支负载。这说明矩阵计算对 L1D 数据供给压力非常明显，部分数据访问还会继续下探到 LLC 或更低层级。Frontend Bound 达到 26.3%，也是五类负载中最高的，说明该负载除了数据供给外，还可能受到循环体指令供给、解码、I-cache 或前端队列影响。

3. 访存密集型负载 read64

read64 的 IPC 为 4.234，非常高。

从前端看，read64 的循环结构非常简单，通常是连续地址读取、累加、地址递增和循环跳转。循环分支简单，分支预测失败率接近 0%，说明控制流非常规律，前端几乎不会因为分支错误而频繁重定向。

从访存子系统看，read64 是大块连续读取，访问模式类似：

buf, buf+8, buf+16, buf+24, ...

这种规律访问非常适合硬件预取器。预取器可以提前把后续 cache line 拉入 L1/L2/LLC，从而隐藏一部分内存访问延迟。因此虽然它是访存密集型负载，但 IPC 仍然很高。

成对指标中，L1D Load Miss Rate 只有 0.039%，说明大量 load 在 L1D 层面被很好地服务；但 LLC Load Miss Rate 为 9.752%，说明当数据规模达到 1GB 后，仍有部分访问会穿透 LLC，需要从内存获取。Topdown 中 Backend Bound 为 23.2%，高于 Frontend Bound 的 7.9%，说明瓶颈主要在后端访存子系统，包括 cache 层级带宽、内存带宽、load buffer 和预取机制。

另外，该负载的 dTLB-load-misses 数量较高，由于它访问的数据量大，页数量多，因此 TLB miss 数会增加；但由于访问是连续的，TLB 局部性仍然较好。

4. 随机访存负载 rand-set

这个负载的核心是：在一块较大的内存区域内随机位置写入/设置数据。随机地址序列缺少稳定步长，硬件预取器难以预测下一次访问位置，cache line 的空间局部性也较差。

rand-set 的 IPC 为 4.881，是五类负载中最高的，但是并不代表他的访存压力小，需要综合其他指标来看。Generic Cache Miss Rate 达到 47.014%，LLC Load Miss Rate 达到 35.602%，远高于 read64。这说明随机访问破坏了空间局部性和时间局部性，硬件预取器很难预测下一次访问地址，LLC 层面的 miss 明显增加。

从后端看，Topdown 中 Backend Bound 为 29.4%，说明随机访存给后端带来了压力。其主要瓶颈可能包括：

随机地址生成，TLB 查询，L1/L2/LLC miss，store buffer 占用，内存访问延迟。

但它的分支预测失败率几乎为 0%，说明控制流仍然很规律。前端只是在持续执行随机地址生成和写入循环，并没有大量不可预测分支。Frontend Bound 为 13.8%，Backend Bound 为 29.4%，说明主要瓶颈仍偏向访存后端，而不是前端取指解码。

它和 read64 的关键区别是：read64 的连续访问可以被预取器有效隐藏延迟，而 rand-set 的随机访问难以预取，cache line 利用率低，因此更容易暴露 LLC miss、store 路径和随机内存延迟。

5. 分支密集型负载 queens

queens 的 IPC 为 1.950，是五类负载中最低的。分支预测失败率为 11.625%，远高于其他负载。这是因为 queens 是 N 皇后回溯搜索问题，核心代码包含大量条件判断、递归/回溯、合法性检

查和分支跳转，性能主要被错误分支预测拖慢。分支预测失败时，CPU 会丢弃错误路径上已经取指、解码甚至进入流水线的指令，并从正确路径重新取指，导致前端重定向和流水线清空，后端执行单元也会出现空泡。

这正是 queens IPC 较低的主要原因。分支预测失败后，CPU 会清空错误路径上的流水线工作，丢弃已经取指、解码甚至部分执行的微操作，然后从正确路径重新取指。即：
前端重定向->pipeline flush->ROB 中错误路径 μ ops 被清除->后端执行单元短暂空转->IPC 下降。

从 Topdown 原始数据看，queens 的 Frontend Bound 为 10.3%，Backend Bound 为 7.7%，两者都不算特别高；真正突出的其实是 topdown-bad-spec，原始数据中 bad speculation 相关计数很大。这与高分支预测失败率一致，说明 queens 的主要瓶颈不是 cache，也不是单纯前端取指不足，而是 错误分支预测导致的大量错误路径执行和流水线恢复开销。

访存方面，queens 的 L1D Load Miss Rate 只有 0.006%，LLC Load Miss Rate 为 2.195%，说明棋盘状态、递归栈、bitmask 等数据大多可以留在 L1/L2 cache 中，访存不是主要瓶颈。

3. 总结

从前端角度看，`matrixprod` 的 Frontend Bound 最高，为 26.3%，说明其指令供给或前端队列存在一定压力；`queens` 的关键问题则是分支预测失败率高达 11.625%，属于控制流和 bad speculation 主导。

从后端执行角度看，`int64` 的 Backend Bound 最高，为 46.3%，主要受整数执行吞吐、端口资源和依赖链影响；`rand-set` 的 Backend Bound 为 29.4%，主要由随机访存和 cache 层级访问延迟造成。

从访存子系统角度看，`matrixprod` 的 L1D Load Miss Rate 最高，说明矩阵计算对 L1D 数据供给最敏感；`rand-set` 的 Generic Cache Miss Rate 和 LLC Load Miss Rate 最高，说明随机访存对 LLC/内存层级压力最大；`read64` 的顺序访问对 L1D 较友好，但由于工作集大，仍会产生大量地址翻译和更低层 cache 访问事件。